HOW I BUILT

Bench Studio

Owning the creative layer instead of renting it: model aggregators, prompt refinement, and the receipts ledger, explained step by step.

Companion guide to the Build Your Own Higgsfield video · Mark Kashef

Free, no email gate · skool.com/earlyaidopters/about

01 - THE LAYER YOU'RE ACTUALLY PAYING FOR

What a creative platform like Higgsfield does behind the Generate button

When you click Generate on a platform like Higgsfield, the interface itself is not making the image or video. Underneath it is a catalog of models built by companies like Google, OpenAI, Black Forest Labs, ByteDance, and Kling. Those models do the heavy lifting. The platform takes the vague thing you typed, turns it into a prompt the selected model is more likely to understand, applies useful defaults, sends the request through an API, stores the result, and converts the provider's price into its own credits or subscription rules.

That work is genuinely useful. A good wrapper saves you from learning every API and every model's quirks. But once you see it as a layer, not magic, you have two simple ways to stop paying a premium for it.

The two paths

- Path one, direct. If you already know the exact model you like and that company offers a first-party API, go straight to it. You pay the model provider, call the model, and remove the extra creative-suite subscription from the chain. This is not zero markup, the provider still sets a price, but you drop one paid wrapper layer.
- Path two, your own aggregator. If you still want a visual interface and a large model catalog, connect one interface to a model API marketplace such as fal.ai. It exposes many image and video models through one API. You curate the endpoints you actually use into your own interface, and pay for individual outputs instead of a monthly bundle.

Bench Studio is Path two, already built, with 37 curated endpoints wired in.

02 - THE ARCHITECTURE

How Bench Studio turns an idea into a receipt

One request flows through four stages: your idea, the capability router, the model provider, and the local archive. The browser never sees provider secrets, it talks to a local loopback API that owns credentials, validates model-specific payloads, streams progress, mirrors the output file, and records durable metadata.

- Idea in: you describe the result in plain English, through the UI or through an MCP-connected agent.
- Capability router: selects a model and inspects its accepted inputs. Every endpoint has different assumptions, some accept one reference image, some accept a list, some require a start frame, some accept none.
- Editable draft: the router returns an editable prompt and a preflight cost estimate before anything is spent.
- Provider execution: on approval, a validated model-specific payload goes to fal.ai (or a direct provider API).
- Local archive: the result is mirrored to disk and logged with model, provider, final prompt, and recorded cost.

This is the same convenience people pay a creative platform for. The difference is that the routing, the prompt layer, the files, and the ledger are all visible and yours.

03 - THE TWO CONNECTION PATHS IN PRACTICE

fal.ai vs direct provider keys, and when to use each

The generate skill and Bench Studio both support two ways to reach the same underlying models. You can mix them, and a direct provider key always wins over fal for its lane, so the receipt stays honest about which route billed you.

Path A: one fal.ai key

A single FAL_KEY unlocks Wan, Hailuo, Kling, and Seedance through one signup, with a small per-clip markup over raw provider price. This is the 60-second setup. Top up $5 and that covers roughly 15 video generations.

```
FAL_KEY=...
```

Path B: individual provider accounts

One account per provider, billed at raw API price, cheapest per clip, most independent from any single aggregator's pricing changes.

- MINIMAX_API_KEY, Hailuo direct, about $0.25/clip, easiest direct signup at platform.minimax.io
- QWEN_CLOUD_API_KEY (or legacy DASHSCOPE_API_KEY), Wan direct plus qwen-image, at qwencloud.com
- KLING_ACCESS_KEY + KLING_SECRET_KEY, Kling direct, requires a Kling global API plan
- Seedance stays fal-only, BytePlus direct API is enterprise-console only

The honest cost comparison

Higgsfield changes its plans, credit allocations, and regional prices. Direct API prices also change by model, duration, resolution, and audio. A test ledger on this system ran $3.88 total against a $79 monthly plan, which is real for that low-volume test, but it is not a universal 3 to 5x claim. If you generate enough on an unlimited plan, the subscription can still beat direct pricing. Put the same model, settings, resolution, and date side by side before you claim a multiplier.

04 - PROMPT REFINEMENT

The model is only half the product

A raw, lazy prompt into a capable model still produces a mediocre result. In a controlled test, the same model (fal-ai/flux-2/flash), the same seed, and the same settings produced two very different outputs depending only on the prompt treatment.

Raw prompt

```
make a premium ad for a matte black perfume bottle called NOCTURNE on wet stone
at night
```

Result: invented a large headline, added a misspelled line, and produced a busier, more template-like composition than intended.

Refined prompt

```
A minimalist matte black perfume bottle with "NOCTURNE" etched in silver,
standing on a glistening wet obsidian rock. Cinematic product photography style.
A dark misty shoreline at midnight. Harsh moonlight rim lighting creates sharp
highlights. Low-angle close-up shot. Palette of #000000 charcoal, #C0C0C0 silver,
and #1A1A1A deep slate.
```

Result: cleaner single-product composition, stronger material separation, restrained hierarchy, and no invented headline. Both generations cost $0.0017 each, verified billed cost, isolated test ledger.

What actually changed

- Material called out explicitly (matte black, etched silver, wet obsidian)
- Camera specified (low-angle close-up shot)
- Lighting specified (harsh moonlight rim lighting)
- Environment specified (dark misty shoreline at midnight)
- Exact color palette given as hex values, not vague adjectives
- No open-ended request for a headline, so the model didn't invent one

This single controlled pair is compelling but not proof that refinement always wins by this margin. Treat it as a demonstrated pattern, not a universal multiplier, and re-test it against your own prompts and models.

05 - THE MCP LAYER

The same backend, reachable from an agent

Bench Studio's MCP server exposes eleven tools over the exact same loopback API the visual interface uses. That means the routing, the prompt layer, the files, and the ledger stay identical whether a human clicks Generate or an agent calls a tool.

- list_models and get_model_capabilities, discover models and inspect each one's accepted inputs before spending anything
- upload_media, send a local reference file and keep both its hosted and local archive URL
- create_media, generate an image or video with an explicit, validated payload
- list_results and get_usage, read back results, previews, and spend without guessing
- create_website, create_document, get_project, list_projects, build and poll a website or a designed PDF

The full step-by-step connector setup for Claude Code, Codex, and Cursor is in mcp_connector_setup.txt in this package.

06 - THE RECEIPTS LEDGER PATTERN

Cost transparency without marketing math

Every model generation should show its exact cost instead of disappearing behind an abstract credit. Bench Studio and the generate skill both implement this the same way: a running markdown or SQLite ledger that records the timestamp, engine, prompt snippet, output file, and estimated or recorded cost for every single generation.

Why this matters

- You can see your real spend at any moment instead of watching a credit meter drop with no unit attached
- Estimated, metered, and recorded costs are kept distinct, never presented as the same fact
- A running total makes it trivial to compare a month of actual usage against a subscription tier before renewing

Minimal ledger row format

```
| Timestamp | Engine | Prompt | Output | Est. cost |
| 2026-08-11 08:56 | qwen-image | a small coral origami crane... | crane.jpg |
$0.030 |
```

Build this into any personal tool the same way: log the row after a successful call, keep a running total at the bottom, and always mark fal-lane or estimate-based costs as estimates rather than confirmed billed amounts.

07 - WHEN TO KEEP THE SUBSCRIPTION

The honest dividing line

Owning the layer is not free of tradeoffs. An all-in-one subscription handles hosting, updates, presets, and support for you. Owning the layer means maintaining a small piece of software yourself.

Keep the subscription if you need:

- Managed identity or character-consistency tools
- A hosted, collaborative workspace for a team
- A large built-in preset library
- Support and mobile access
- Enough monthly volume that an unlimited plan genuinely beats per-generation pricing

Try the owned path if you're mainly paying for:

- Access to models you could call directly or through a marketplace like fal.ai
- Prompt polish you could replicate with a per-model prompt skill
- A clean Generate button and not much else

The models are doing the heavy lifting either way. The wrapper makes them convenient. Now you know exactly what that convenience layer does, two ways to replace it, and what an honest cost comparison actually requires.

KEEP BUILDING YOUR OWN STACK

Want to learn the fundamentals so you can build your own creative infrastructure like Bench Studio?

Inside the Early AI Dopters community, you get the full ***LIVING*** Claude Code Magic course:

- The plugin building blocks (manifests, slash commands, hooks, state files) taught from zero, the same primitives that power Bench Studio
- The exact patterns I used to build Bench Studio and the generate skill, walked through line by line in a dedicated module
- Direct access to me and my team of coaches when you hit a wall setting up your own stack
- A working group of builders shipping their own creative infrastructure variants

Join here: skool.com/earlyaidopters/about